

CMPT 743 Practices for Visual Computing

Ali Mahdavi Amiri

Matrix

- A set of vectors (rows, columns)

$$\begin{pmatrix} a_{10} & a_{12} & \cdots & a_{1m} \\ a_{20} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n0} & a_{n1} & \cdots & a_{nm} \end{pmatrix}_{n \times m}$$

Linear Dependency

- A set of vectors $v_1, v_2, \dots, v_n$ are linearly dependent if there exist non-zero scalars $\alpha_1, \alpha_2, \dots, \alpha_n$ such that:

$$\alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n = 0$$

Linear Dependency

- A set of vectors $v_1, v_2, \dots, v_n$ are linearly dependent if there exist non-zero scalars $\alpha_1, \alpha_2, \dots, \alpha_n$ such that:

$$\alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n = 0$$

$$v_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad v_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad v_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Linear Dependency

- A set of vectors $v_1, v_2, \dots, v_n$ are linearly dependent if there exist non-zero scalars $\alpha_1, \alpha_2, \dots, \alpha_n$ such that:

$$\alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n = 0$$

$$v_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad v_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad v_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\alpha_0 = -1, \alpha_1 = 1, \alpha_2 = 1$$

Rank

- An $n \times m$ matrix has rank r if the maximum number of linearly independent columns or rows is r .

Rank

- An $n \times m$ matrix has rank r if the maximum number of linearly independent columns or rows is r .
- The rank of an $n \times m$ matrix is at most $\min(n, m)$.

Range Space

- Any linear combination of the column vectors of a matrix is called its column space or its range.

$$A \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} a_{11}v_1 & \cdots & a_{1n}v_n \\ \vdots & \cdots & \vdots \\ a_{m1}v_1 & \cdots & a_{mn}v_n \end{bmatrix} = v_1 \begin{bmatrix} a_{11} \\ \vdots \\ a_{m1} \end{bmatrix} + \cdots + v_n \begin{bmatrix} a_{1n} \\ \vdots \\ a_{mn} \end{bmatrix}$$

Range Space

- Any linear combination of the column vectors of a matrix is called its column space or its range.

$$A \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} a_{11}v_1 & \cdots & a_{1n}v_n \\ \vdots & \cdots & \vdots \\ a_{m1}v_1 & \cdots & a_{mn}v_n \end{bmatrix} = v_1 \begin{bmatrix} a_{11} \\ \vdots \\ a_{m1} \end{bmatrix} + \cdots + v_n \begin{bmatrix} a_{1n} \\ \vdots \\ a_{mn} \end{bmatrix}$$

- If $A = \begin{bmatrix} 2 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}$ then the range of A is $\begin{bmatrix} 2v_1 \\ -1v_2 \\ v_1 \end{bmatrix}$.

Null Space

- If $A\mathbf{v} = 0$ and $\mathbf{v} \neq 0$ then $\mathbf{v} \in \text{null}(A)$.

Null Space

- If $A\mathbf{v} = 0$ and $\mathbf{v} \neq 0$ then $\mathbf{v} \in \text{null}(A)$.
- If A is a square matrix and it has null space, then it is a singular matrix.

Span

- The span of a set of vectors is the set of all points obtainable by linear combination of the original vectors.

Span

- The span of a set of vectors is the set of all points obtainable by linear combination of the original vectors.
- For instance vectors $[1,0,0]$, $[0,1,0]$, $[0,0,1]$ span the entire 3D space. However $[1,0,0]$ and $[0,1,0]$ only span XY plane.

Span

- The span of a set of vectors is the set of all points obtainable by linear combination of the original vectors.
- If we have n linearly independent vectors, we can span R^n .

Eigen Decomposition

- An **eigenvector** of a square matrix A is a non-zero vector $\boldsymbol{v}$ such that:

$$A\boldsymbol{v} = \lambda\boldsymbol{v}$$

Eigen Decomposition

- An eigenvector of a square matrix A is a non-zero vector $\boldsymbol{v}$ such that:

$$A\boldsymbol{v} = \lambda\boldsymbol{v}$$

- The scalar λ is the **eigenvalue** of matrix A .

Eigen Decomposition

- Consider matrix A has n linearly independent eigenvectors $\{v_1, \dots, v_n\}$ with corresponding eigenvalues $\{\lambda_1, \dots, \lambda_n\}$.

Eigen Decomposition

- Consider matrix A has n linearly independent eigenvectors $\{v_1, \dots, v_n\}$ with corresponding eigenvalues $\{\lambda_1, \dots, \lambda_n\}$. If we concatenate all the eigenvectors to form a matrix $V = [v_1, \dots, v_n]$ and also concatenate all the eigenvalues to form $\lambda = [\lambda_1, \dots, \lambda_n]$.

Eigen Decomposition

- Consider matrix A has n linearly independent eigenvectors $\{v_1, \dots, v_n\}$ with corresponding eigenvalues $\{\lambda_1, \dots, \lambda_n\}$. If we concatenate all the eigenvectors to form a matrix $V = [v_1, \dots, v_n]$ and also concatenate all the eigenvalues to form $\lambda = [\lambda_1, \dots, \lambda_n]$. The eigendecomposition of A is given by:

$$A = V \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_n \end{bmatrix} V^{-1}$$

Eigen Decomposition

- Special Case: A real symmetric matrix A has form

$$A = Q \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_n \end{bmatrix} Q^T$$

- Q is an orthogonal matrix whose columns are the eigenvectors of A . Columns of Q span R^n .

Determinant

- The determinant of a square matrix is a function that maps matrices to real scalars.

$$A = \begin{bmatrix} 4 & -2 \\ 3 & 6 \end{bmatrix} \Rightarrow |A| = (4)(6) - (-2)(3) = 30$$

Determinant

- The determinant of a square matrix is a function that maps matrices to real scalars.
- A matrix is singular if and only if its determinant is zero.

Determinant

- The determinant of a square matrix is a function that maps matrices to real scalars.
- A matrix is singular if and only if its determinant is zero.
- In fact, the determinant is equal to the product of all the eigenvalues of the matrix.

Characteristic Map

- The following statements are equivalent for a square matrix A :
 - λ is an eigenvalue of A .
 - $\det(A - \lambda I) = 0$

Characteristic Map

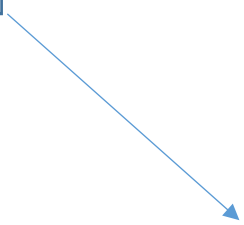
- The following statements are equivalent for a square matrix A :
 - λ is an eigenvalue of A .
 - $\det(A - \lambda I) = 0$ Why?

Characteristic Map

- The following statements are equivalent for a square matrix A :

- λ is an eigenvalue of A .

- $\det(A - \lambda I) = 0$

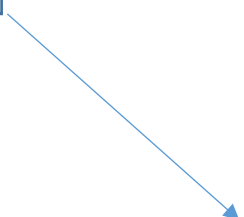


We can use it to find eigenvalues of matrix A

Characteristic Map

- The following statements are equivalent for a square matrix A :
 - λ is an eigenvalue of A .

- $\det(A - \lambda I) = 0$



Example

We can use it to find eigenvalues of matrix A

Trace

- The trace operator gives the sum of all the diagonal entries of a matrix:

$$\text{Tr}(A) = \sum_i A_{i,i}$$

Trace

- The trace operator gives the sum of all the diagonal entries of a matrix:

$$\text{Tr}(A) = \sum_i A_{i,i}$$

Ex.

$$A = \begin{bmatrix} 4 & -2 \\ 3 & 6 \end{bmatrix}$$

$$\text{Tr}(A) = 10$$

Expected Value

- Let X be a random variable with a finite number of outcomes $x_1, x_2, \dots, x_k$ occurring with probabilities $p_1, p_2, \dots, p_k$, respectively. The expectation of X is defined as:

$$E[x] = \sum_{i=1}^k x_i p_i$$

Expected Value

- Let X be a random variable with a finite number of outcomes $x_1, x_2, \dots, x_k$ occurring with probabilities $p_1, p_2, \dots, p_k$, respectively. The expectation of X is defined as:

$$E[x] = \sum_{i=1}^k x_i p_i$$

- In case of events with equal probability $E[X]$ is the average of X .

Expected Value

- Let X be a random variable with a finite number of outcomes $x_1, x_2, \dots, x_k$ occurring with probabilities $p_1, p_2, \dots, p_k$, respectively. The expectation of X is defined as:

$$E[x] = \sum_{i=1}^k x_i p_i$$

- In case of events with equal probability $E[X]$ is the average of X .
- Properties: $E[aX] = aE[X]$ if $a \in R^{m \times n}$

$$E[Xa] = E[X]a \text{ if } a \in R^{p \times n}$$

Variance

- The variance of a random variable X is the expected value of the squared deviation from the mean of X , $\mu = E[X]$

$$Var[X] = E[(x - \mu)^2]$$

Co-Variance

- The covariance between two random variables X and Y is defined as:

$$Cov(X, Y) = E[(X - E[X])(Y - E[Y])^T]$$

- Properties:

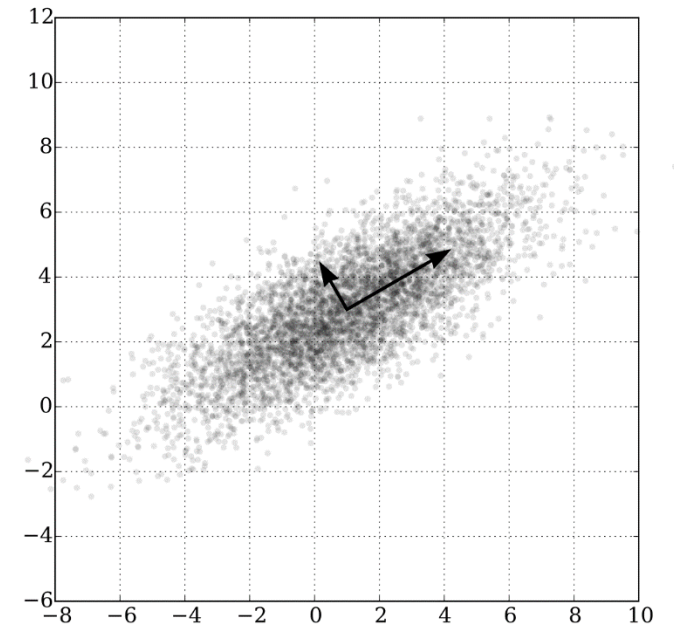
$$Cov(X, X) = Var(X)$$

PCA

- Principle Component Analysis

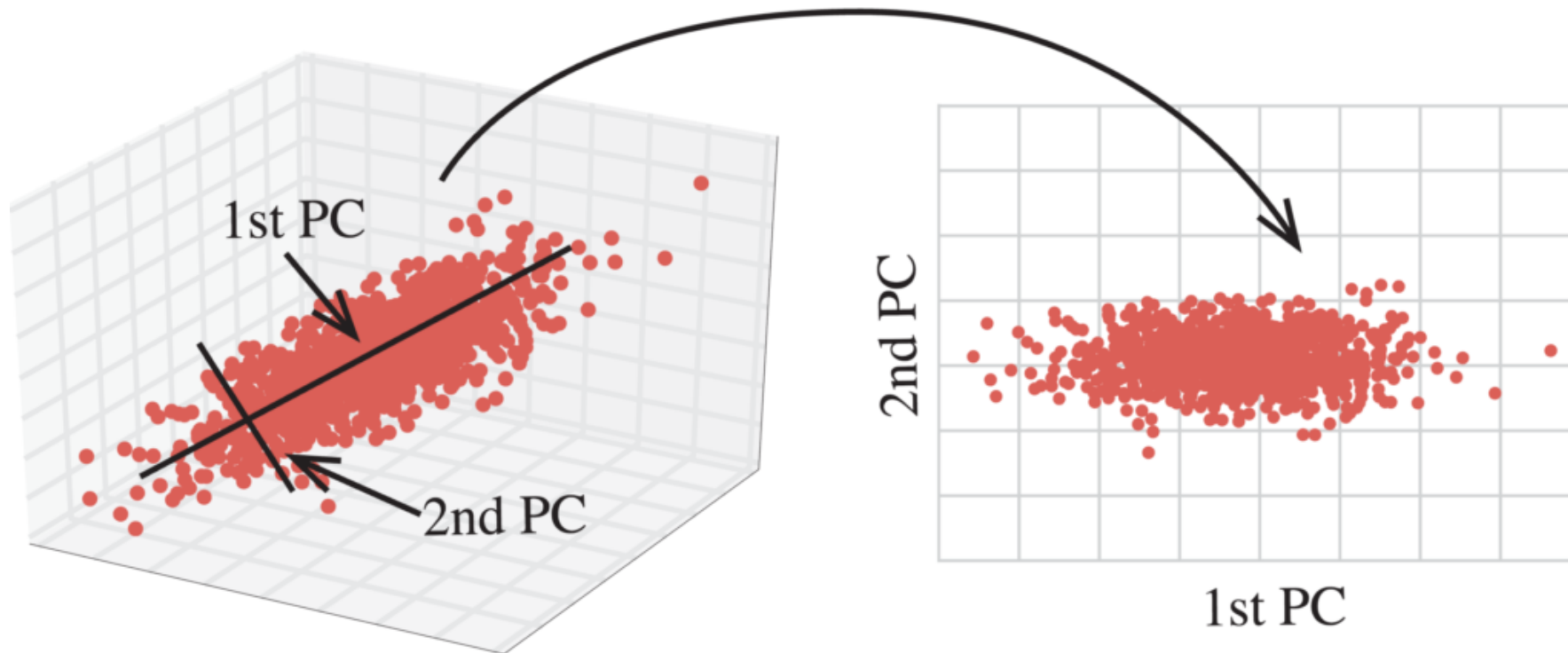
PCA

- Principle Component Analysis
- This is a method to find the trend of the data



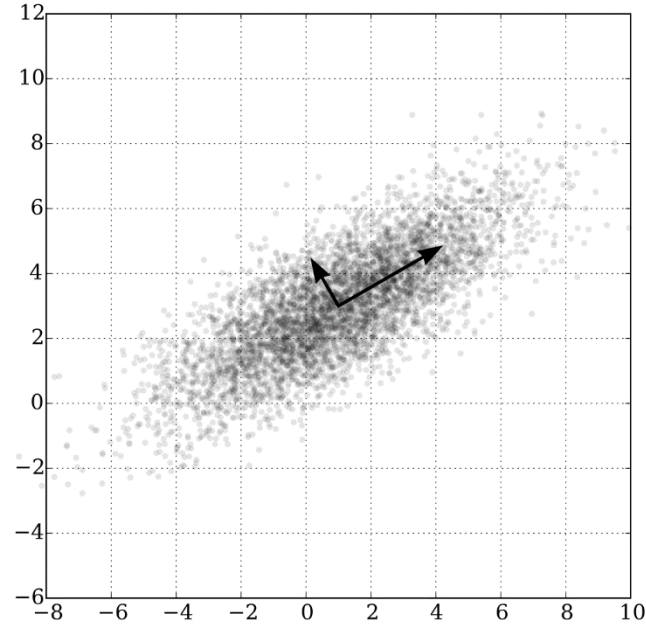
PCA

- PCA can be used in dimensionality reduction, compression and many other applications.



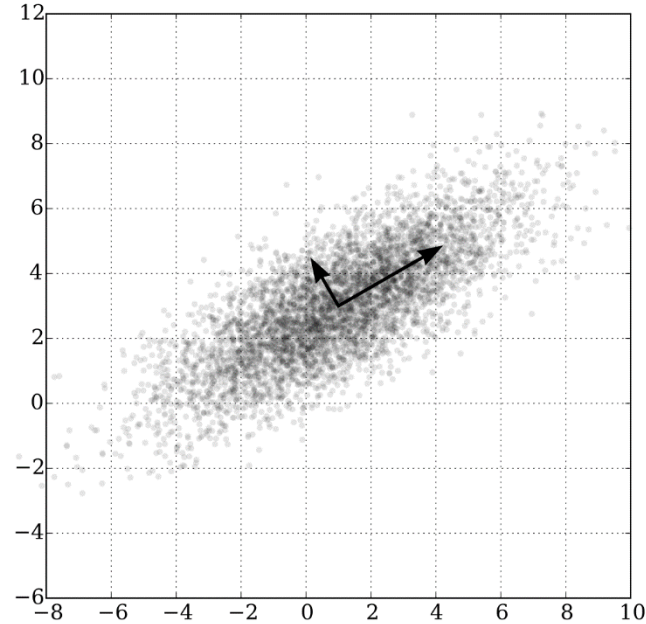
PCA

- How can we define and find these trends of the data?



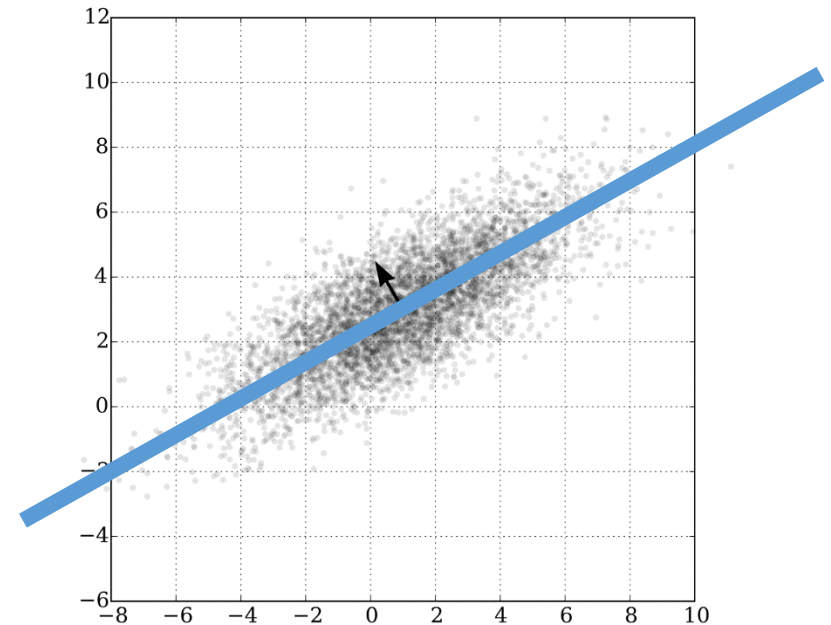
PCA

- What do we mean by the trend of the data?



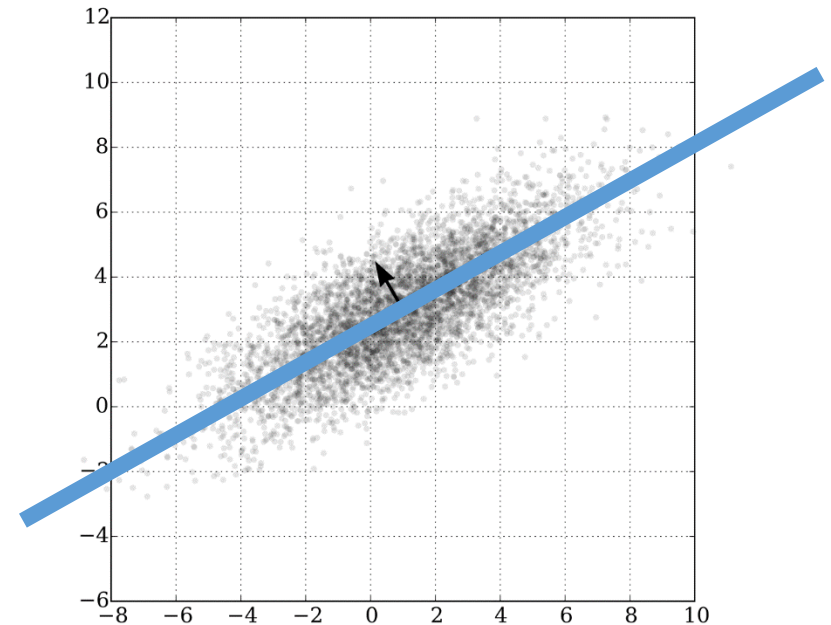
PCA

- We are looking for a vector



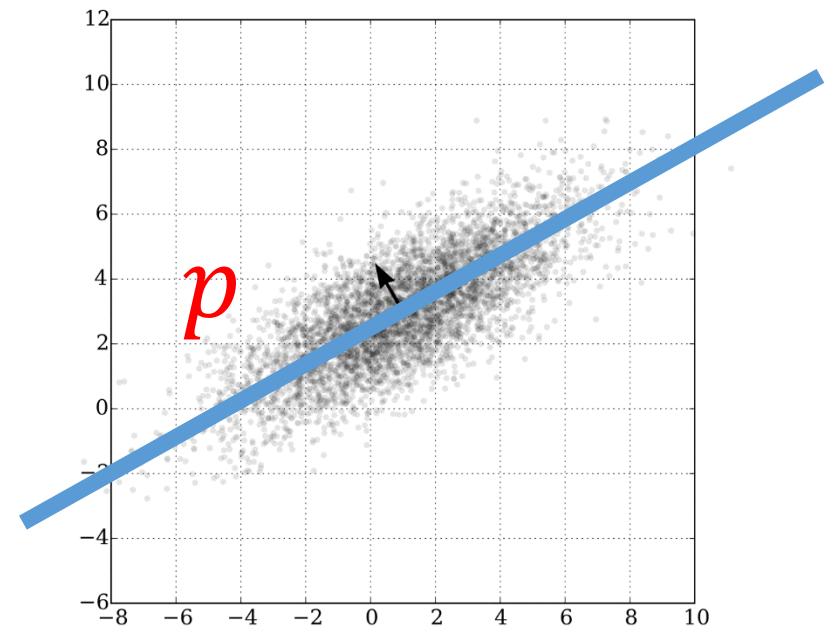
PCA

- We are looking for a vector on which the **variance** of the data is maximum.



PCA

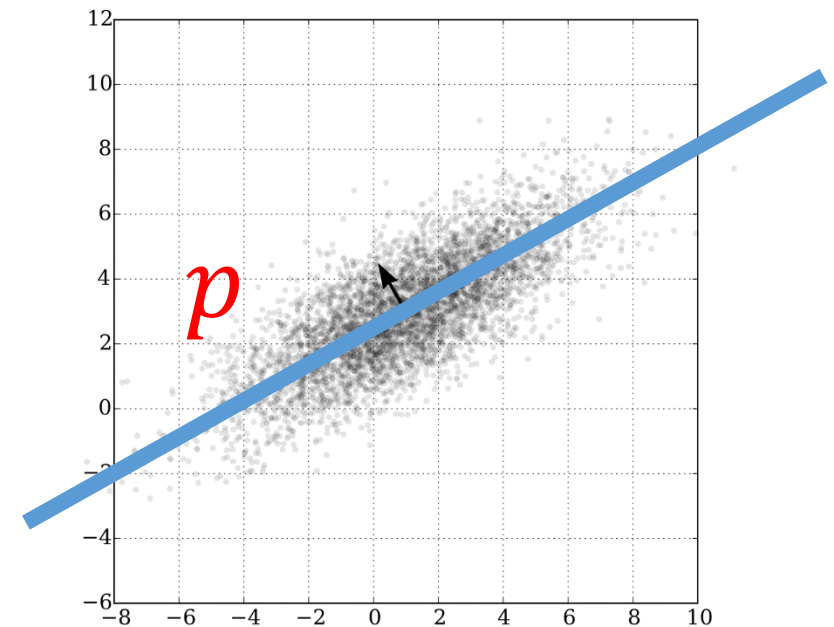
- Data is p ;



PCA

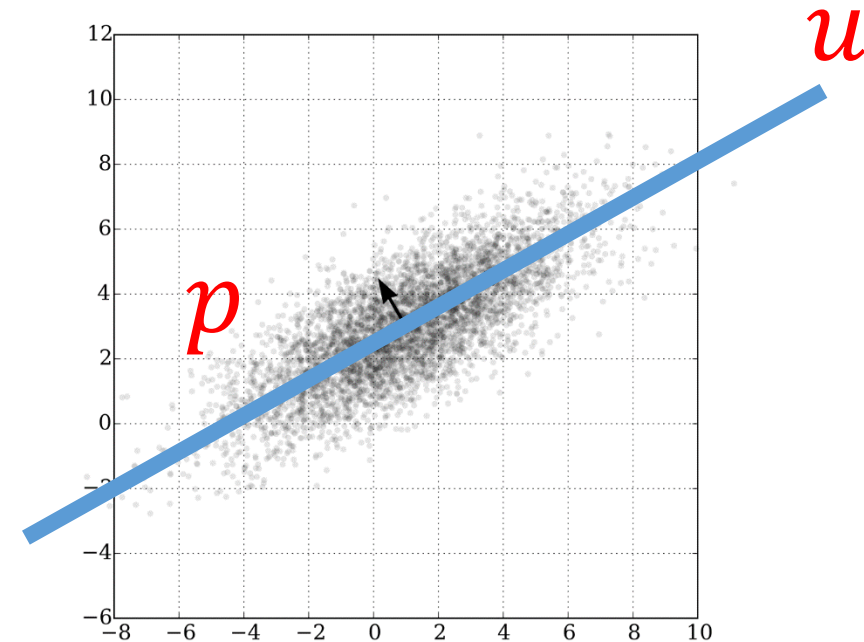
- Data is p ;
- Example

$$p = \begin{bmatrix} | & & | \\ p_1 & \ddots & p_n \\ | & & | \end{bmatrix} = \begin{bmatrix} x_1 & & x_n \\ y_1 & \ddots & y_n \\ z_1 & & z_n \end{bmatrix}$$



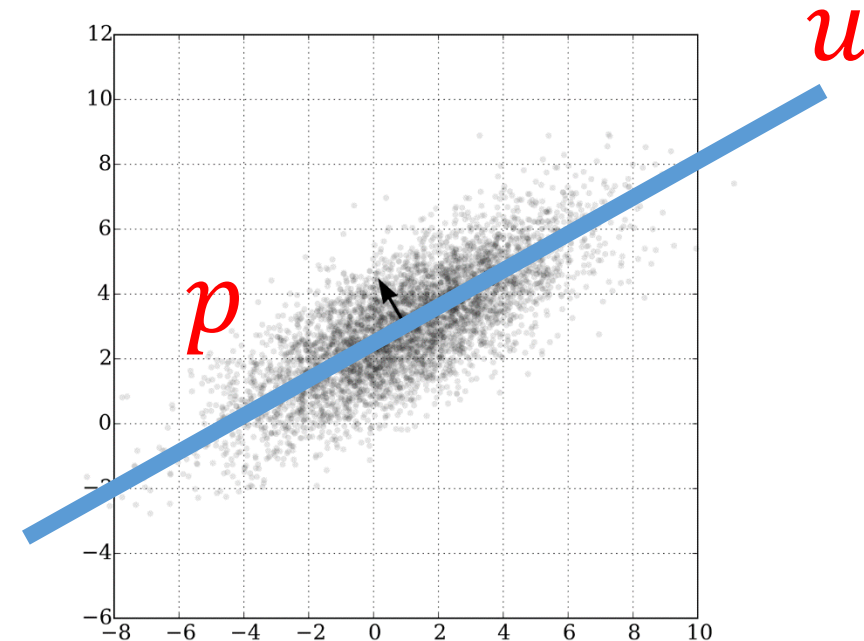
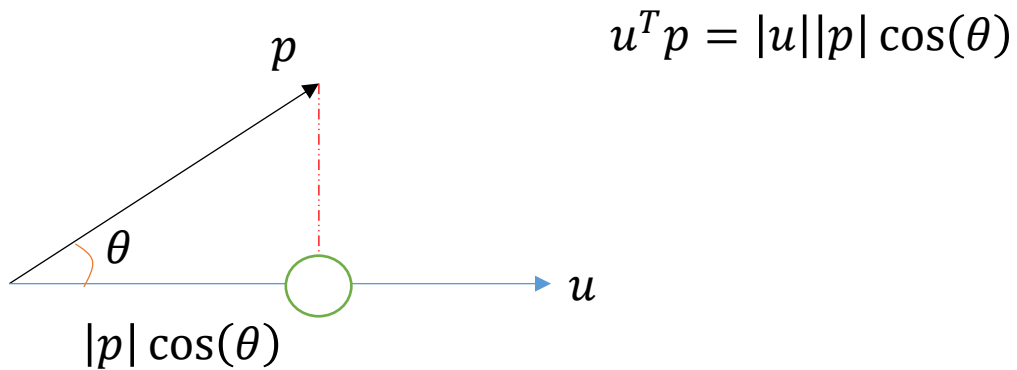
PCA

- Data is p ;
- Its component along a proposed axis u is $(u^T p)$



PCA

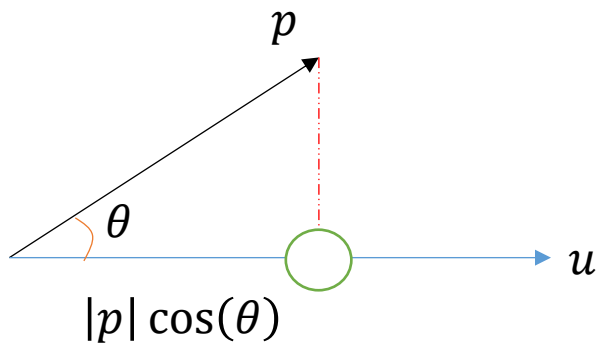
- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- Reminder of projection:



PCA

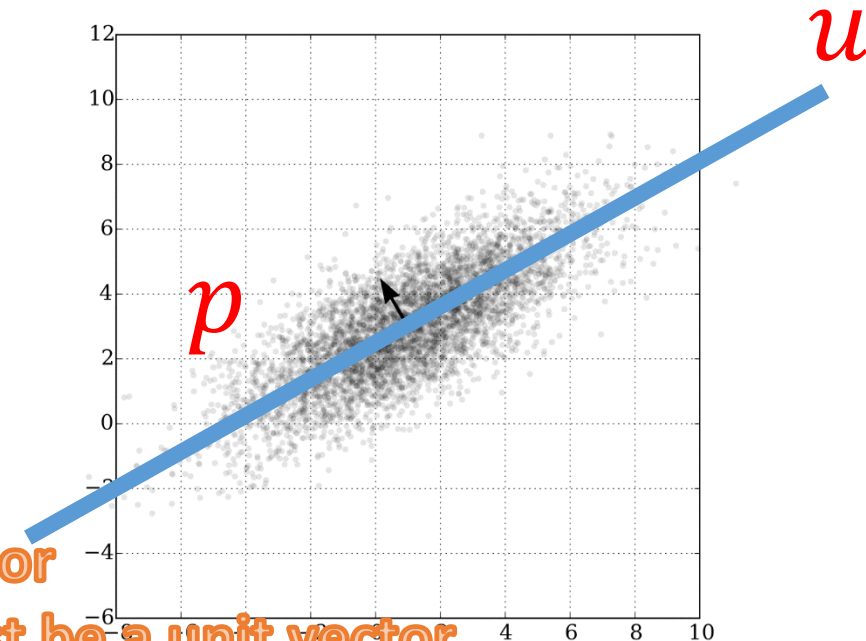
- Data is p ;
- Its component along a proposed axis u is $(u^T p)$

- Reminder of projection:



$$u^T p = |u| |p| \cos(\theta)$$

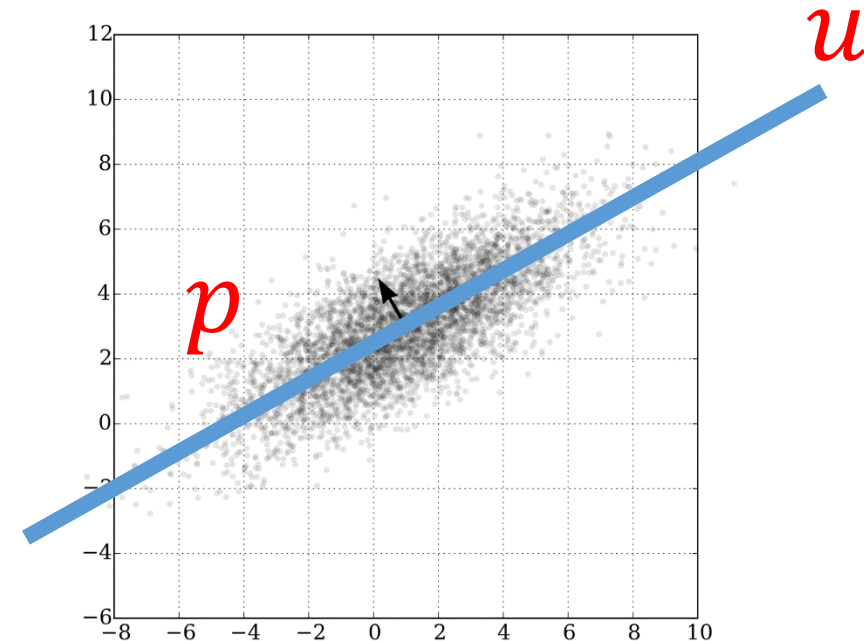
$$u^T p = |p| \cos(\theta)$$



If u is a unit vector
Later we will discuss why u must be a unit vector

PCA

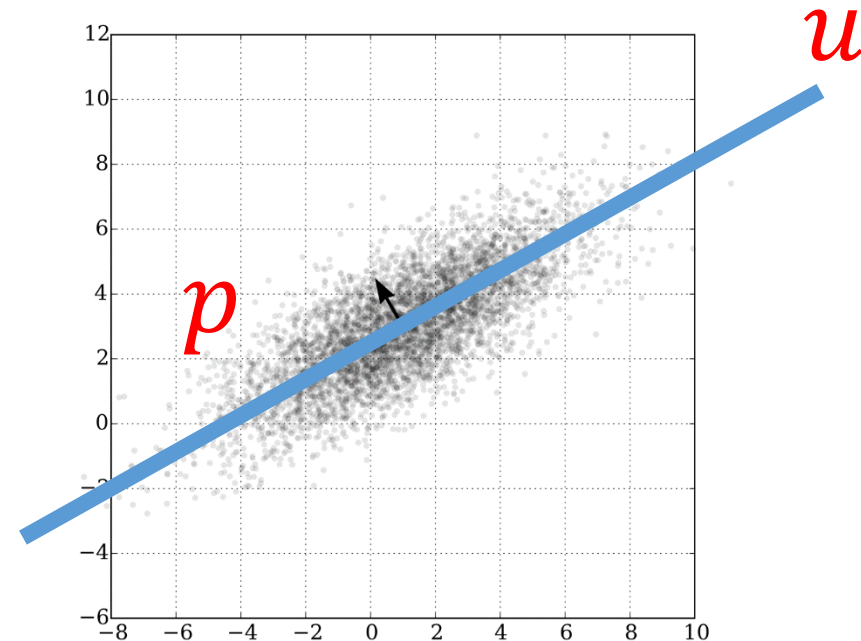
- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- The variance of $u^T p$ is $E(u^T p - E(u^T p))^2$



PCA

- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- Its variance is $E(u^T p - E(u^T p))^2$

$$E(u^T p - E(u^T p))^2 = E[(u^T (p - E(p)))^2]$$

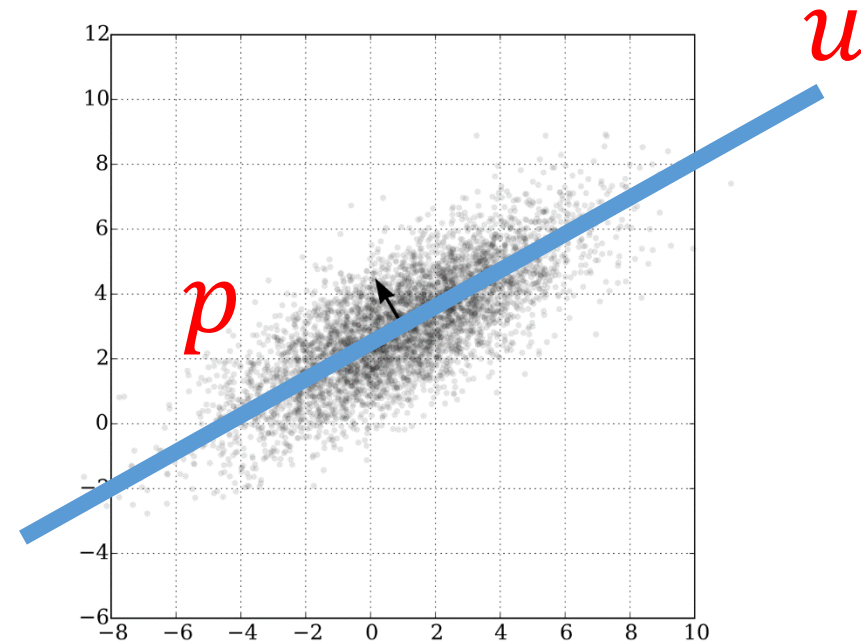


PCA

- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- Its variance is $E(u^T p - E(u^T p))^2$

$$E(u^T p - E(u^T p))^2 = E[(u^T (p - E(p)))^2]$$

What is $E(p)$?

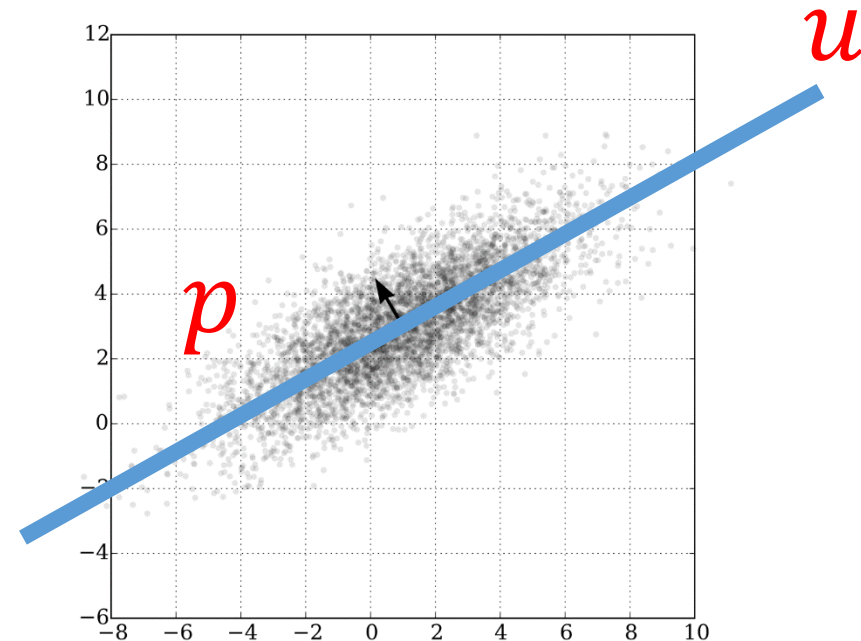


PCA

- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- Its variance is $E(u^T p - E(u^T p))^2$

$$E(u^T p - E(u^T p))^2 = E[(u^T (p - E(p)))^2]$$

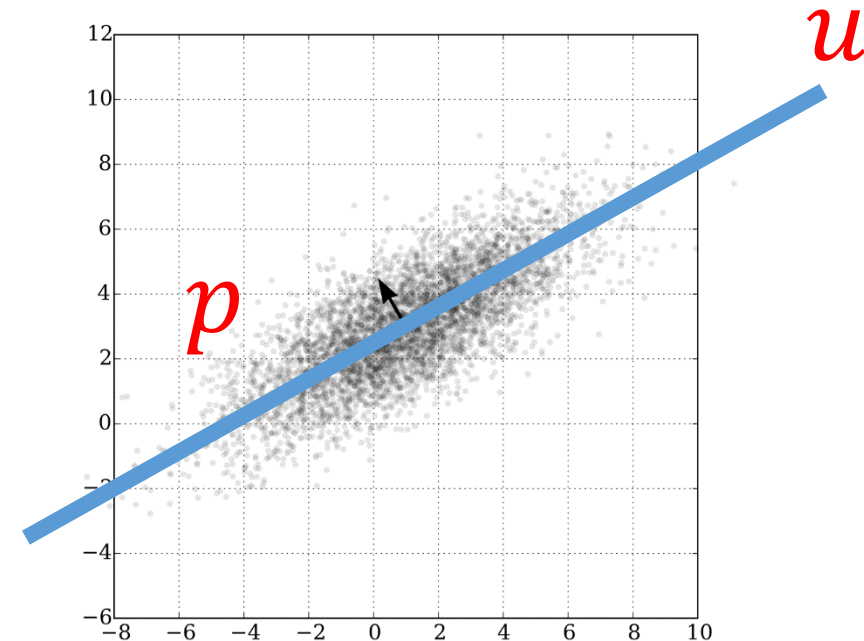
Average of p or center of mass



PCA

- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- Its variance is $E(u^T p - E(u^T p))^2$

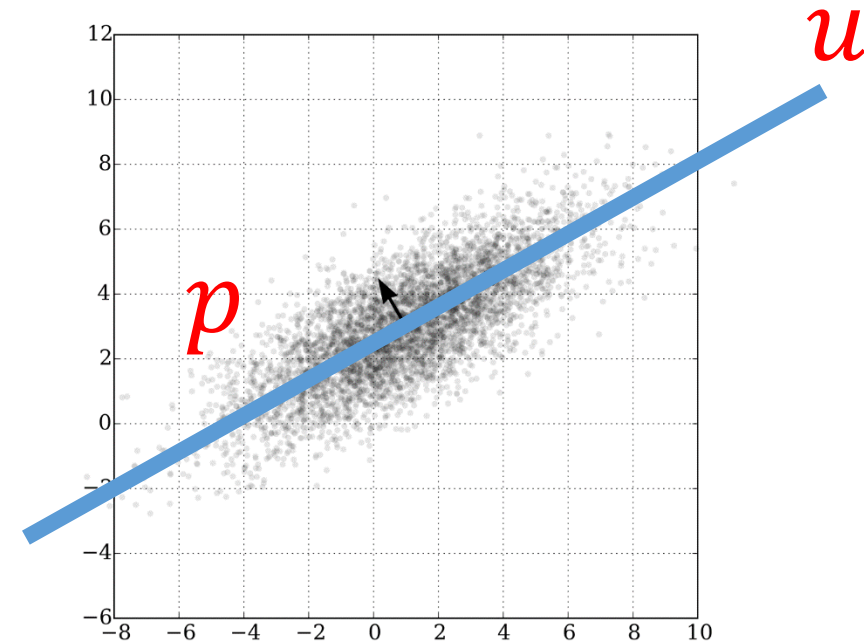
$$\begin{aligned} E(u^T p - E(u^T p))^2 &= E[(u^T (p - E(p)))^2] \\ &= u^T E[(p - E(p))(p - E(p))^T] u \end{aligned}$$



PCA

- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- Its variance is $E(u^T p - E(u^T p))^2$

$$\begin{aligned} E(u^T p - E(u^T p))^2 &= E[(u^T (p - E(p)))^2] \\ &= u^T E[(p - E(p)) \cdot (p - E(p))^T] u \\ &= u^T C u \end{aligned}$$

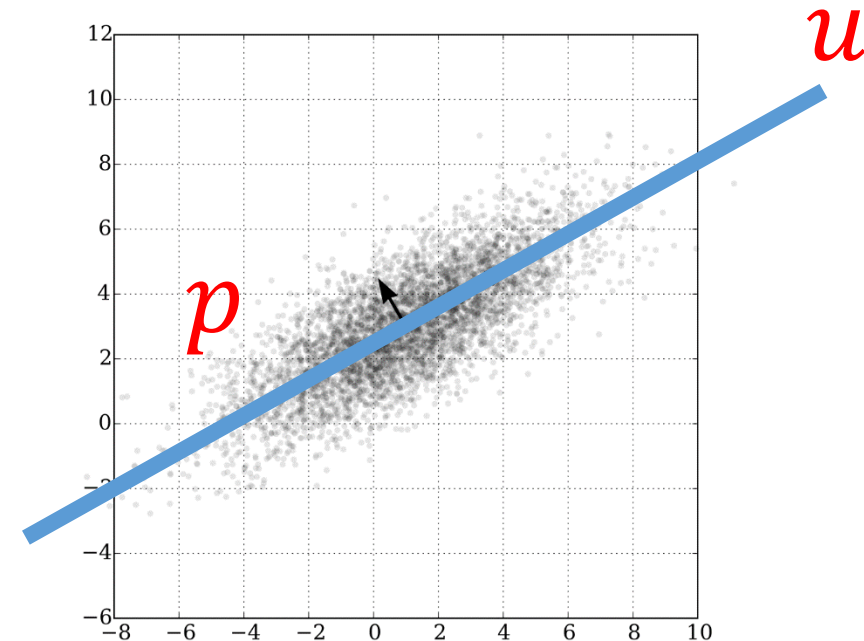


PCA

- Data is p ;
- Its component along a proposed axis u is $(u^T p)$
- Its variance is $E(u^T p - E(u^T p))^2$

$$\begin{aligned} E(u^T p - E(u^T p))^2 &= E[(u^T (p - E(p)))^2] \\ &= u^T E[(p - E(p)) \cdot (p - E(p))^T] u \\ &= u^T C u \end{aligned}$$

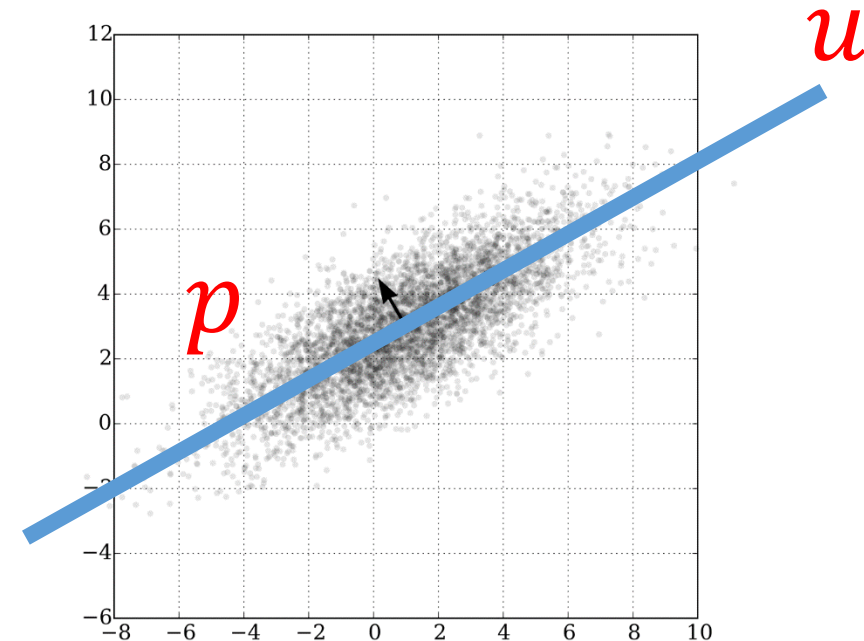
Co-variance Matrix



PCA

- We are looking for a vector u that maximizes this relationship

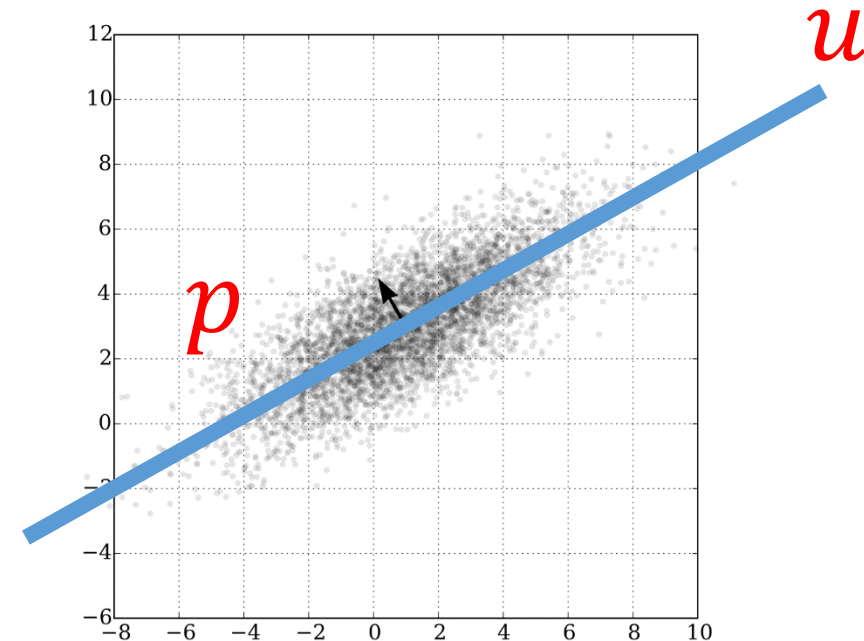
$$\begin{aligned} E(u^T p - E(u^T p))^2 &= E[(u^T (p - E(p)))^2] \\ &= u^T E[(p - E(p))(p - E(p))^T] u \\ &= u^T C u \end{aligned}$$



PCA

- We are looking for a vector u that maximizes this relationship
- We also assume that u is a unit vector: $u^T u = 1$

$$\begin{aligned} E(u^T p - E(u^T p))^2 &= E[(u^T (p - E(p)))^2] \\ &= u^T E[(p - E(p))(p - E(p))^T] u \\ &= u^T C u \end{aligned}$$

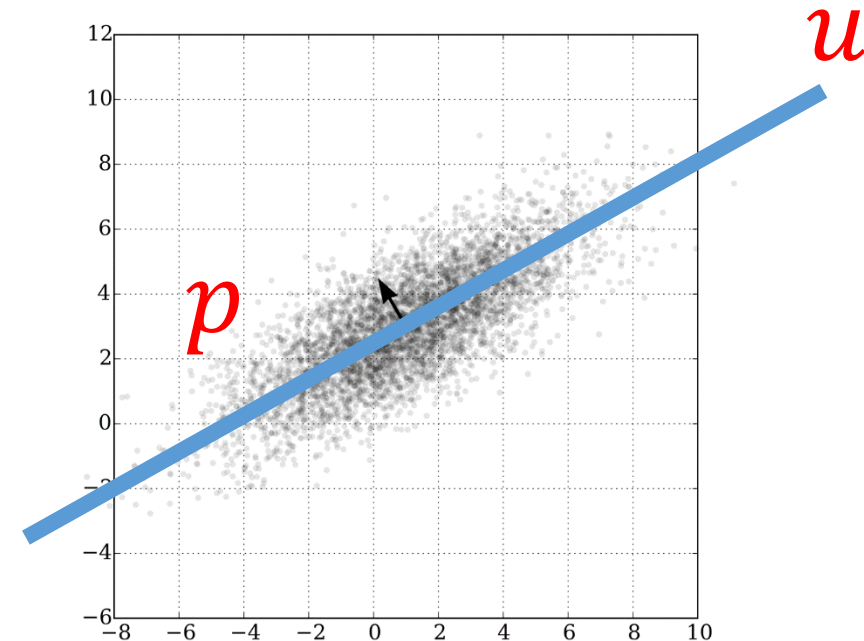


PCA

- We are looking for a vector u that maximizes this relationship
- We also assume that u is a unit vector: $u^T u = 1$

$$\begin{aligned} E(u^T p - E(u^T p))^2 &= E[(u^T (p - E(p)))^2] \\ &= u^T E[(p - E(p))(p - E(p))^T] u \\ &= u^T C u \end{aligned}$$

Why?



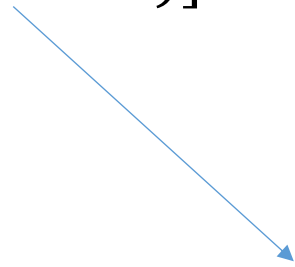
PCA

- To maximize $u^T C u$ subject to $u^T u = 1$ we use Lagrange multiplier

$$\text{Max} [u^T C u - \lambda(u^T u - 1)]$$

PCA

- To maximize $u^T C u$ subject to $u^T u = 1$ we use Lagrange multiplier

$$\text{Max} [u^T C u - \lambda(u^T u - 1)]$$


If $u^T u$ deviates from 1 by a huge factor,
the whole term $(u^T C u - \lambda(u^T u - 1))$ becomes small

PCA

- To maximize $u^T C u$ subject to $u^T u = 1$ we use Lagrange multiplier

$$\text{Max } [u^T C u - \lambda(u^T u - 1)]$$

- We need to take the derivative with respect to u to find the maximizer.

PCA

- To maximize $u^T C u$ subject to $u^T u = 1$ we use Lagrange multiplier

$$\text{Max} [u^T C u - \lambda(u^T u - 1)]$$

- Derivative:

$$\frac{d}{du} \text{Max} [u^T C u - \lambda(u^T u - 1)] = 0$$

PCA

- To maximize $u^T C u$ subject to $u^T u = 1$ we use Lagrange multiplier

$$\text{Max} [u^T C u - \lambda(u^T u - 1)]$$

- Derivative:

$$\frac{d}{du} \text{Max} [u^T C u - \lambda(u^T u - 1)] = 0$$

$$C u - \lambda u = 0$$

PCA

- To maximize $u^T C u$ subject to $u^T u = 1$ we use Lagrange multiplier

$$\text{Max} [u^T C u - \lambda(u^T u - 1)]$$

- Derivative:

$$\frac{d}{du} \text{Max} [u^T C u - \lambda(u^T u - 1)] = 0$$

$$C u - \lambda u = 0$$

$$C u = \lambda u$$

PCA

- To maximize $u^T C u$ subject to $u^T u = 1$ we use Lagrange multiplier

$$\text{Max } [u^T C u - \lambda(u^T u - 1)]$$

- Derivative:

$$\frac{d}{du} \text{Max } [u^T C u - \lambda(u^T u - 1)] = 0$$

$$C u - \lambda u = 0$$

$$C u = \lambda u$$

What is this?

PCA

- Which eigenvector should we choose?

PCA

- Which eigenvector should we choose?
- We are looking for maximization of $u^T C u$

PCA

- Which eigenvector should we choose?
- We are looking for maximization of $u^T C u$
- We found that $C u = \lambda u$

PCA

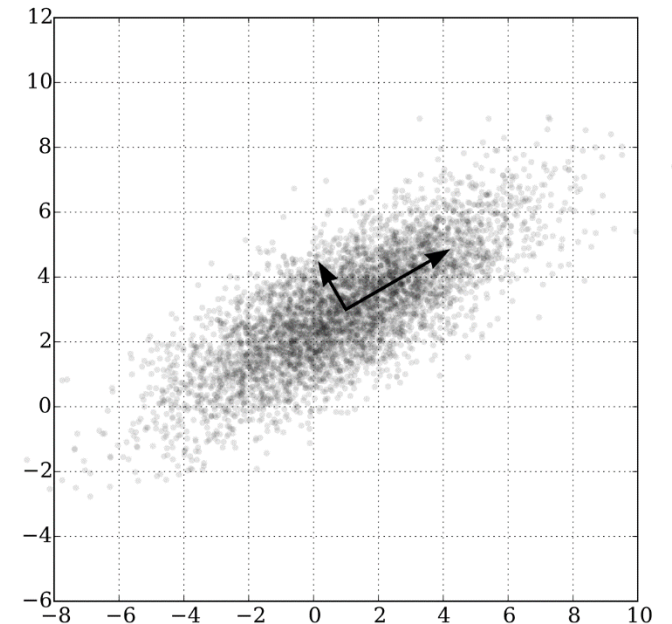
- Which eigenvector should we choose?
- We are looking for maximization of $u^T C u$
- We found that $C u = \lambda u$
- Multiply both sides in $u^T \rightarrow u^T C u = \lambda u^T u = \lambda$

PCA

- Multiply both sides in $u^T \rightarrow u^T C u = \lambda u^T u = \lambda$
- If we want to maximize it, we should pick the largest eigenvalue and its associated eigenvector.

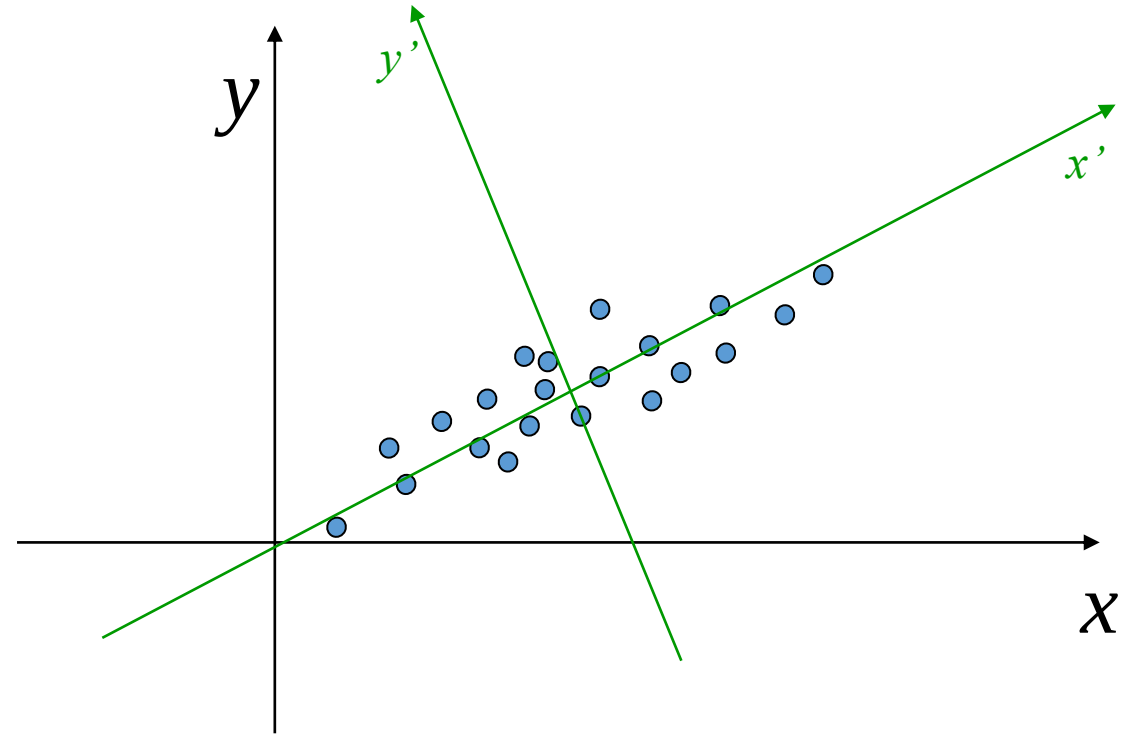
PCA

- The same thing is true for other axes of the trends in the data.
- We can pick second, third, ..., eigenvalues and their associated eigenvectors.



PCA Algorithm

- We wanted to find principal components

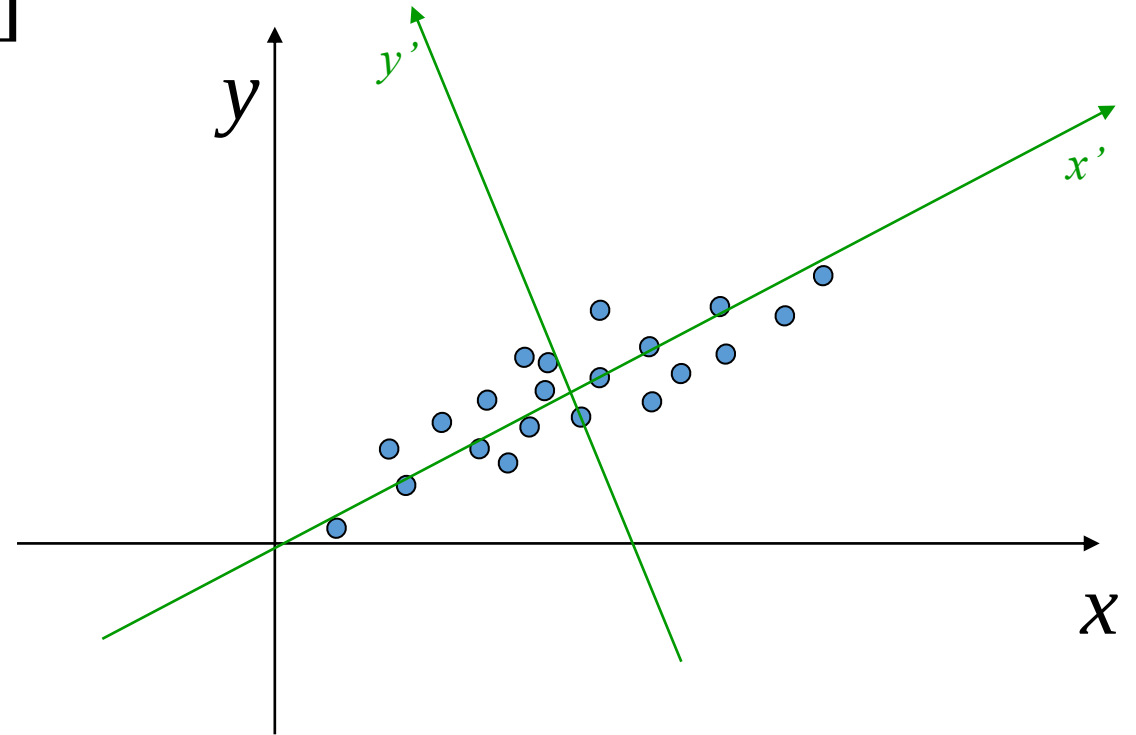


PCA Algorithm

- Move the center of mass to the origin

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

Center of the mass: $m = \frac{1}{n} \sum p_i$



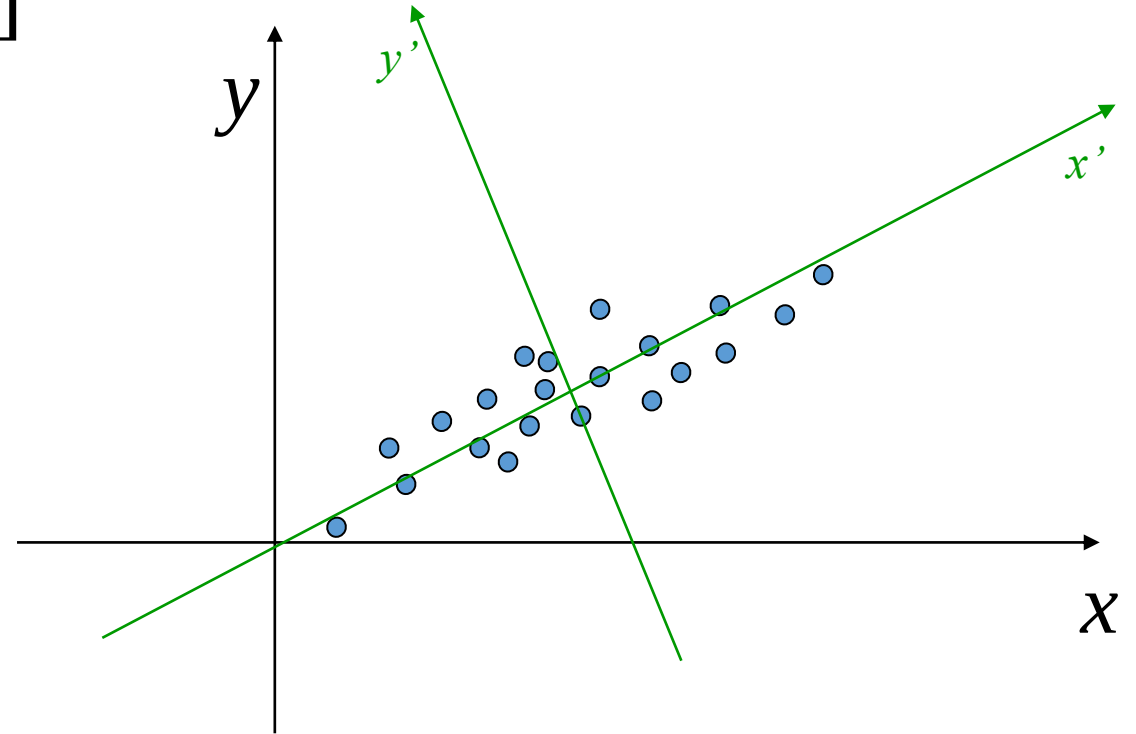
PCA Algorithm

- Move the center of mass to the origin

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

Center of the mass: $m = \frac{1}{n} \sum p_i$

- Move center of the mass to origin



PCA Algorithm

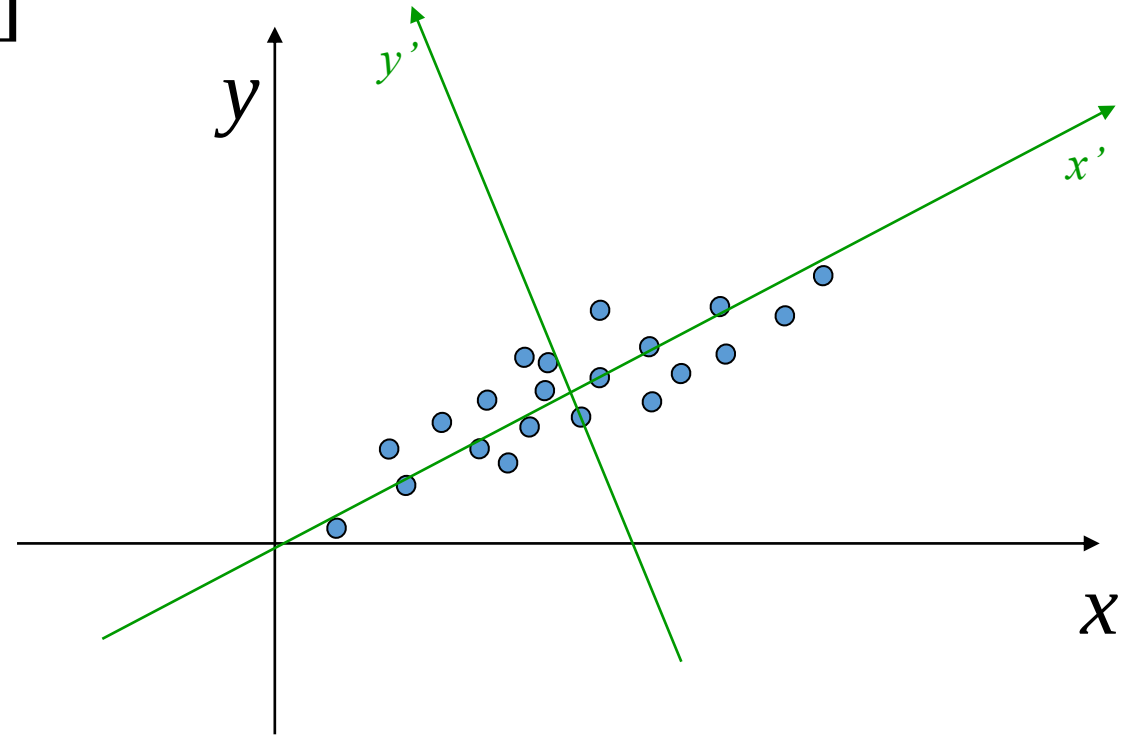
- Move the center of mass to the origin

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

Center of the mass: $m = \frac{1}{n} \sum p_i$

- Move center of the mass to origin

$$\hat{P}_i = P_i - m$$



PCA Algorithm

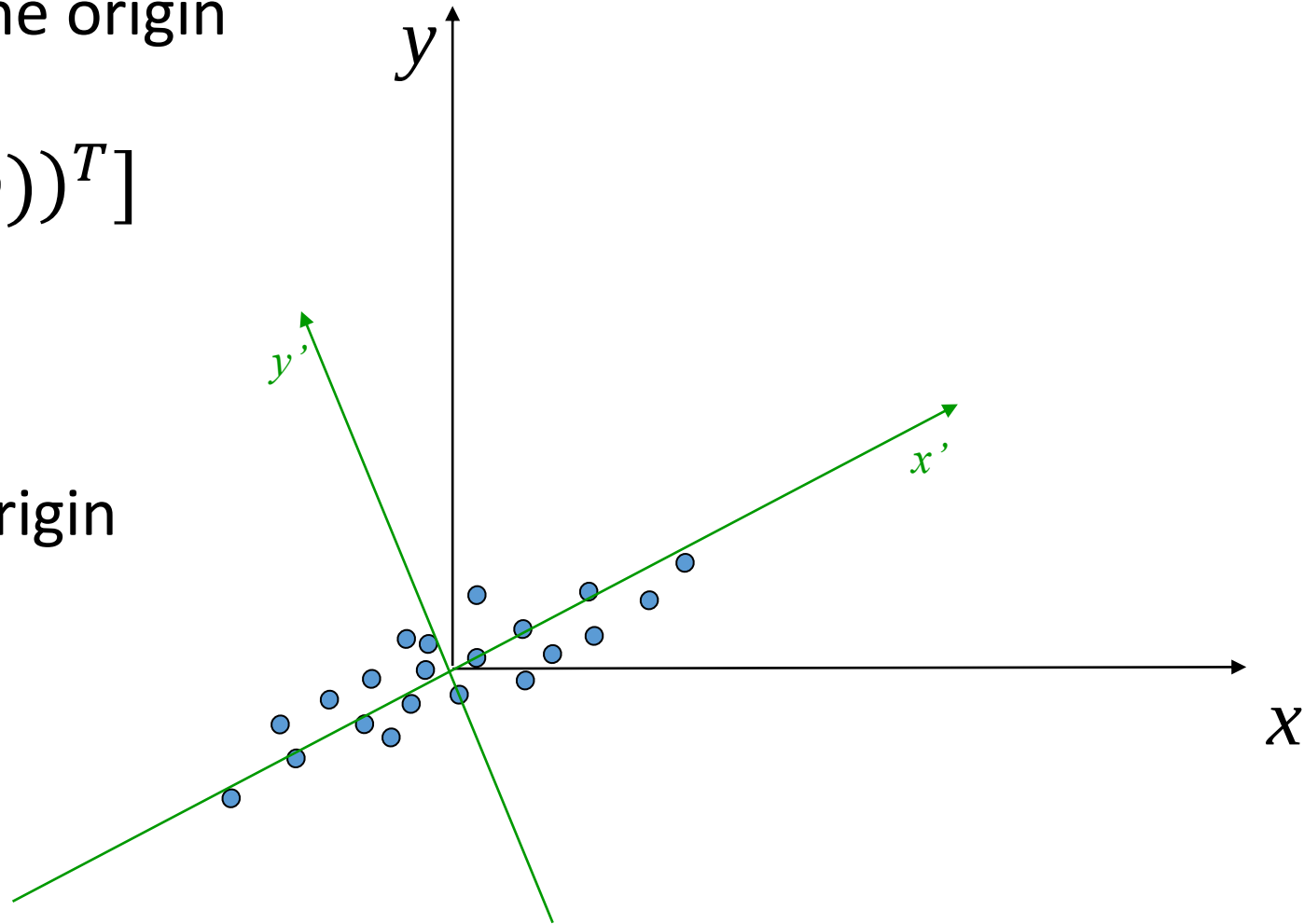
- Move the center of mass to the origin

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

Center of the mass: $m = \frac{1}{n} \sum p_i$

- Move center of the mass to origin

$$\hat{P}_i = P_i - m$$



PCA Algorithm

$$C = E[(p - E(p)).(p - E(p))^T]$$

- Construct the matrix X of the translated data points.

$$X = \begin{bmatrix} | & & | \\ p'_1 & \ddots & p'_n \\ | & & | \end{bmatrix} = \begin{bmatrix} x'_1 & & x'_n \\ y'_1 & \ddots & y'_n \\ z'_1 & & z'_n \end{bmatrix}$$

PCA Algorithm

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

- Construct the matrix X of the translated data points.

$$X = \begin{bmatrix} | & & | \\ p'_1 & \ddots & p'_n \\ | & & | \end{bmatrix} = \begin{bmatrix} x'_1 & & x'_n \\ y'_1 & \ddots & y'_n \\ z'_1 & & z'_n \end{bmatrix}$$

- Construct the Covariance Matrix

$$C = \frac{1}{n} X X^T$$

PCA Algorithm

- The principal axes are the eigenvectors of $C = \frac{1}{n}XX^T$

$$\frac{1}{n}XX^T = S = U \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_d \end{bmatrix} U^T$$

PCA Algorithm

- The principal axes are the eigenvectors of $C = \frac{1}{n}XX^T$

$$\frac{1}{n}XX^T = S = U \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_d \end{bmatrix} U^T$$

Why can we have such a property?

Co-variance Matrix

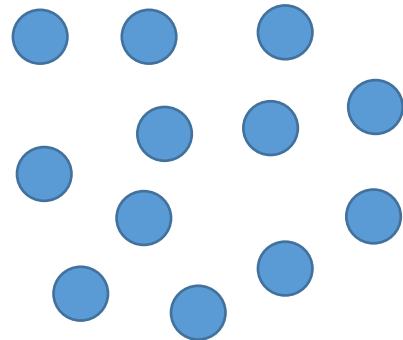
- What is the interpretation of the covariance matrix?

$$C = E[(p - E(p)). (p - E(p))^T]$$

Co-variance Matrix

- What is the interpretation of the covariance matrix?

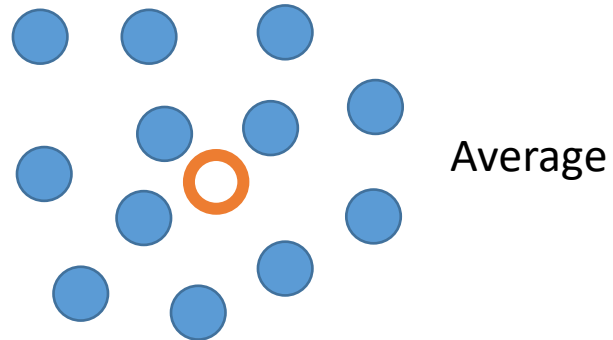
$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$



Co-variance Matrix

- What is the interpretation of the covariance matrix?

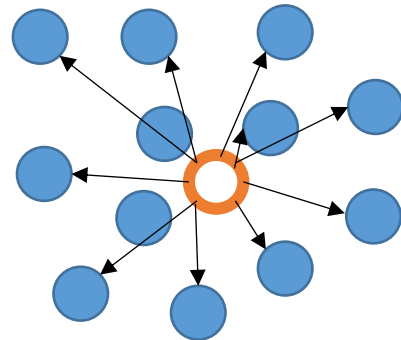
$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$



Co-variance Matrix

- What is the interpretation of the covariance matrix?

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

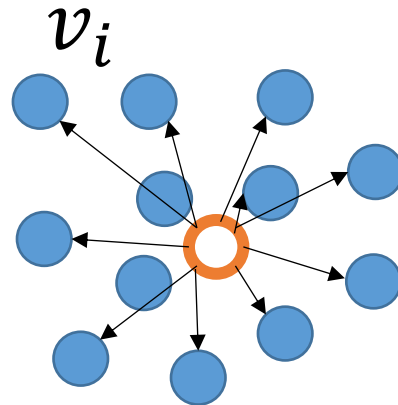


Vectors from average to each data point

Co-variance Matrix

- What is the interpretation of the covariance matrix?

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$



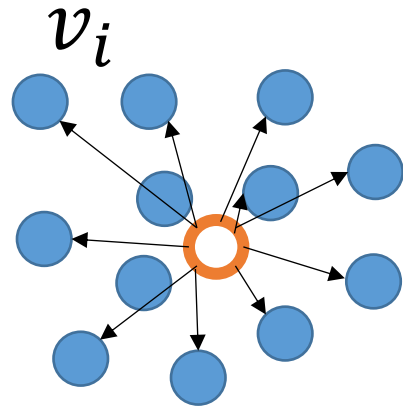
Vectors from average to each data point

Co-variance Matrix

- What is the interpretation of the covariance matrix?

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

We can form a matrix



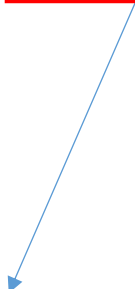
$$X = \begin{pmatrix} v_{1x} & & v_{nx} \\ v_{1y} & \ddots & v_{ny} \\ v_{1z} & & v_{nz} \end{pmatrix}$$

Co-variance Matrix

- What is the interpretation of the covariance matrix?

$$C = E[(p - E(p)) \cdot (p - E(p))^T]$$

We can form a matrix



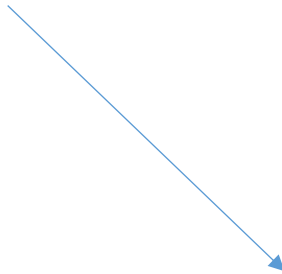
$$\frac{1}{n}XX^T = \frac{1}{n} \begin{pmatrix} v_{1x} & & v_{nx} \\ v_{1y} & \ddots & v_{ny} \\ v_{1z} & & v_{nz} \end{pmatrix} \begin{pmatrix} v_{1x} & v_{1y} & v_{1z} \\ & \ddots & \\ v_{nx} & & v_{nz} \end{pmatrix}$$

Co-variance Matrix

- What is the interpretation of the covariance matrix?

$$C = E[(p - E(p)).(p - E(p))^T]$$

We can form a matrix


$$\frac{1}{n}XX^T = \frac{1}{n} \begin{pmatrix} v_{1x} & & v_{nx} \\ v_{1y} & \ddots & v_{ny} \\ v_{1z} & & v_{nz} \end{pmatrix} \begin{pmatrix} v_{1x} & v_{1y} & v_{1z} \\ & \ddots & \\ v_{nx} & & v_{nz} \end{pmatrix} = \begin{pmatrix} cov(x, x) & cov(x, y) & cov(x, z) \\ cov(y, x) & cov(y, y) & cov(y, z) \\ cov(z, x) & cov(z, y) & cov(z, z) \end{pmatrix}$$

Co-variance Matrix

- What is the interpretation of the covariance matrix?

$$C = E[(p - E(p)).(p - E(p))^T]$$

We can form a matrix

$$\frac{1}{n}XX^T = \frac{1}{n} \begin{pmatrix} v_{1x} & & v_{nx} \\ v_{1y} & \ddots & v_{ny} \\ v_{1z} & & v_{nz} \end{pmatrix} \begin{pmatrix} v_{1x} & v_{1y} & v_{1z} \\ & \ddots & \\ v_{nx} & & v_{nz} \end{pmatrix} = \begin{pmatrix} cov(x, x) & cov(x, y) & cov(x, z) \\ cov(y, x) & cov(y, y) & cov(y, z) \\ cov(z, x) & cov(z, y) & cov(z, z) \end{pmatrix}$$

Real symmetric matrix

Co-variance Matrix

- Therefore, we have such decomposition:

$$\frac{1}{n}XX^T = S = U \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_d \end{bmatrix} U^T$$

Dimensionality Reduction

- Let's consider data point p_i

Dimensionality Reduction

- Let's consider data point p_i
- We can rewrite p_i as:

$$p_i = m + \alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_n u_n$$

Dimensionality Reduction

- Let's consider data point p_i
- We can rewrite p_i as:

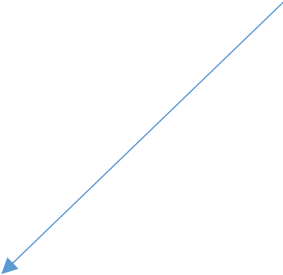
$$p_i = m + \alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_n u_n$$

Why?

Dimensionality Reduction

- Let's consider data point p_i
- We can rewrite p_i as:

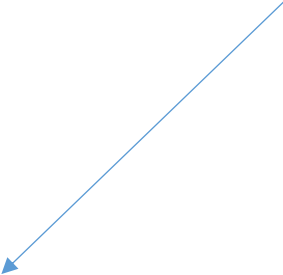
$$p_i = m + \boxed{\alpha_1} u_1 + \alpha_2 u_2 + \cdots + \alpha_n u_n$$


$$\alpha_1 = u_1^T (p_i - m)$$

Dimensionality Reduction

- Let's consider data point p_i
- We can rewrite p_i as:

$$p_i = m + \boxed{\alpha_1} u_1 + \alpha_2 u_2 + \cdots + \alpha_n u_n$$


$$\alpha_1 = u_1^T (p_i - m)$$

Why?

Dimensionality Reduction

- Let's consider data point p_i

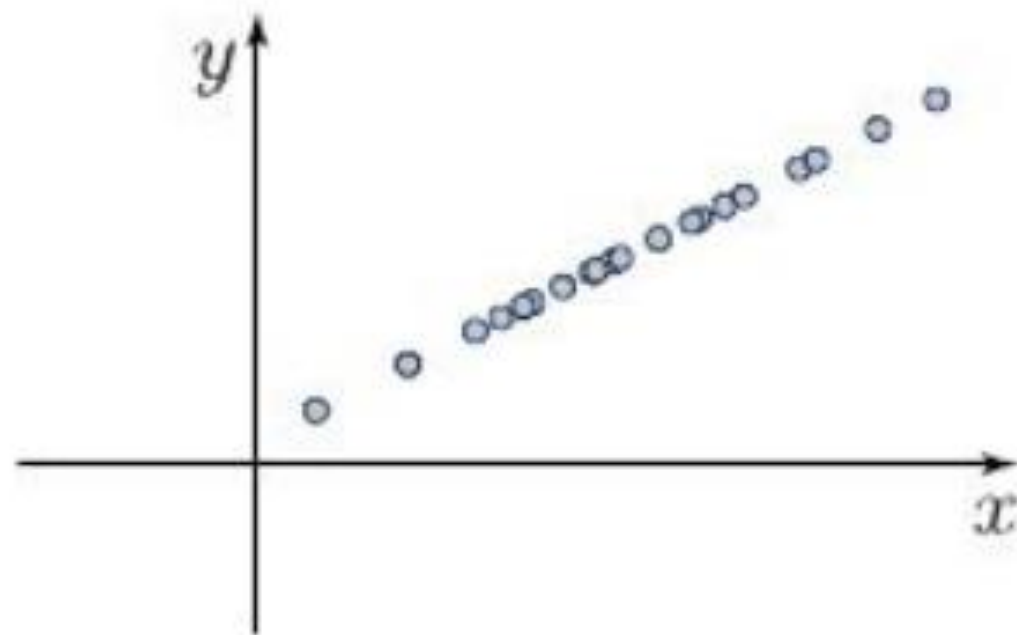
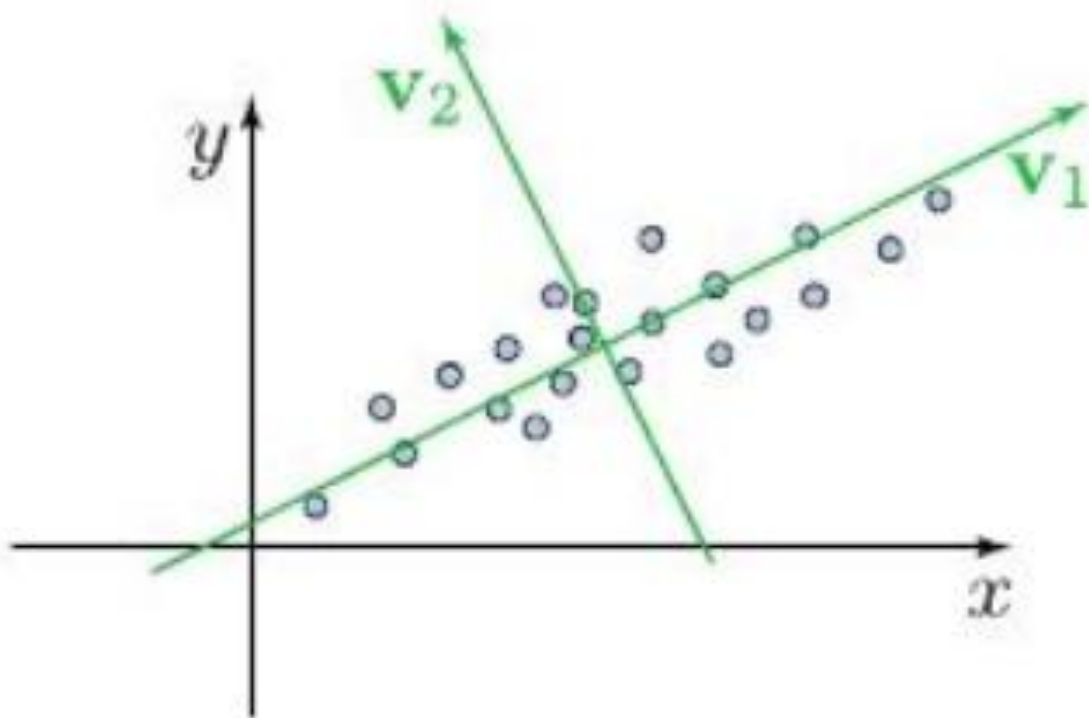
- We can rewrite p_i as:

$$p_i = m + \alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_n u_n$$

- If $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ and u_i are associated eigenvectors to these eigenvalues, then we can reduce the dimension by removing the last few eigenvectors since they have smaller effect on data.

$$p_i = m + \alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_k u_k \quad k < n$$

Dimensionality Reduction



Face Detection

- A sample application of PCA can be Face detection

Face Detection

- It is a hard problem to hardcode or handcraft.



Face Detection

- Sometimes we see some faces where there is no face.



Face Detection

- We want to use PCA to learn the subspace of faces and use it to represent and detect faces.

Face Space

- EigenFace: construct a low dimensional linear sub-space that contain most of the images with low error.

Face Space

- EigenFace: construct a low dimensional linear sub-space that contain most of the images with low error.
- If we use PCA, each face is a vector p



p

We can flatten the image into a vector

Face Space

- If we use PCA, each face is a vector p
- We can find the average face



p



m

Face Space

- If we use PCA, each face is a vector p



p

- We can find the average face, form the covariance matrix and find the eigenvectors (eigenfaces)



m

u_1

u_2

u_3

u_4

u_5

u_6

u_7



Face Space

- We can approximate each face as a combination of the average face and a linear combination of eigenfaces.



The diagram illustrates the Face Space equation. It shows a target face p (leftmost image) and an approximation p' (second image from left). These are equal to the average face m (third image from left) plus a linear combination of seven eigenfaces u_1 through u_7 (images in the row to the right). The eigenfaces are shown as a row of seven small face images, each representing a different feature (e.g., eyes, nose, mouth, etc.).

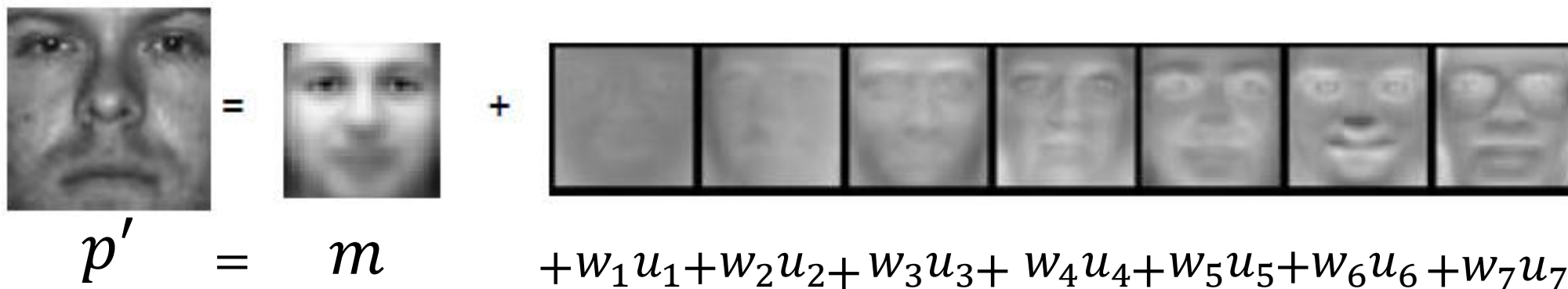
$$p \approx p' = m + w_1 u_1 + w_2 u_2 + w_3 u_3 + w_4 u_4 + w_5 u_5 + w_6 u_6 + w_7 u_7$$

Face Space

- For face detection, we select a windows from an image and it is reconstructed using PCA.

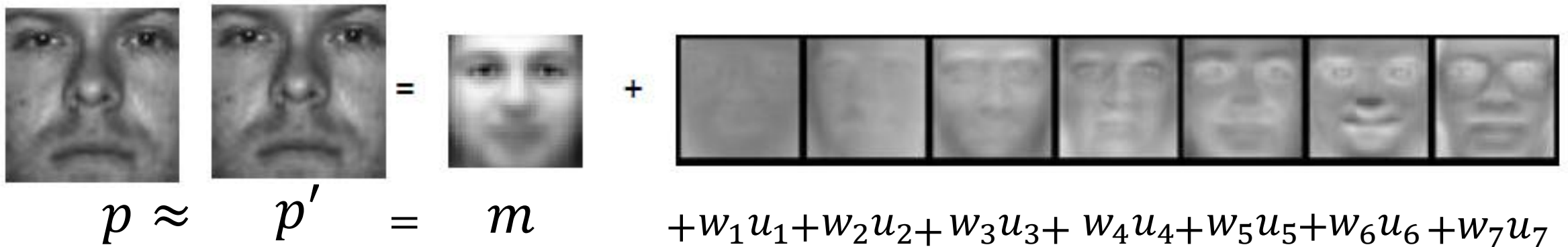


p


$$p' = m + w_1 u_1 + w_2 u_2 + w_3 u_3 + w_4 u_4 + w_5 u_5 + w_6 u_6 + w_7 u_7$$
The diagram shows a target face image p' on the left, followed by an equals sign, a mean face image m , another equals sign, a plus sign, and a row of seven small face images representing principal components u_1 through u_7 . Below the row of images is the mathematical expression for the reconstruction: $+w_1 u_1 + w_2 u_2 + w_3 u_3 + w_4 u_4 + w_5 u_5 + w_6 u_6 + w_7 u_7$.

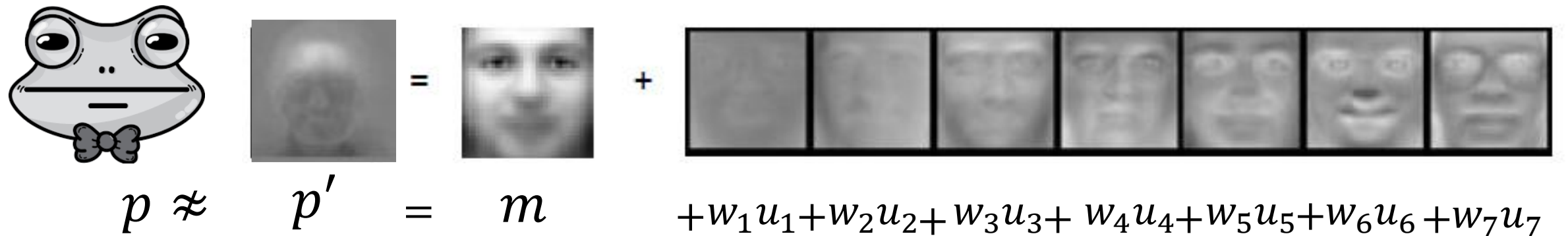
Face Space

- For face detection, we select a windows from an image and it is reconstructed using PCA.
- If $p \approx p'$, the window is face


$$p \approx p' = m + w_1u_1 + w_2u_2 + w_3u_3 + w_4u_4 + w_5u_5 + w_6u_6 + w_7u_7$$

Face Space

- For face detection, we select a windows from an image and it is reconstructed using PCA.
- If $p \approx p'$, the window is face, otherwise it is not


$$p \not\approx p' = m + w_1u_1 + w_2u_2 + w_3u_3 + w_4u_4 + w_5u_5 + w_6u_6 + w_7u_7$$